Control Flow

14

Our brain analyses data throughout each day. This data is made up of the inputs that our body receives from the environment. Actions are performed by our brain depending upon this data. For example, if we feel thirsty, we drink water. Or if the traffic light is green, only then we cross the road. These can be called control flow statements.

Similarly, a computer also performs actions based on control flow statements. It can evaluate the conditions and then decide the results.

Computers can perform data analysis with the help of programs. So, we need to understand how to control the execution flow of a program in Python.

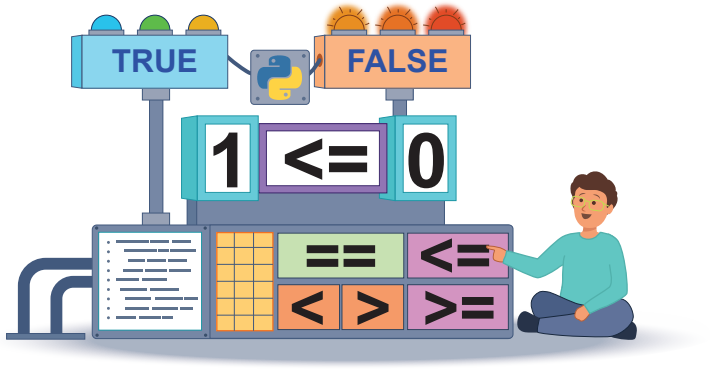
14. Control Flow

Recall:

Data analysis is a process used to evaluate data and improve the productivity of a process. In this process, data is extracted from various sources, cleaned, and categorised.

Control flow

A program's control flow is the order in which the program's instructions are executed. The control flow of a Python program is primarily regulated by conditional statements and loops.



To understand and apply control flow statements, we need to first understand comparison operators used to check conditions.

Comparison operators

We are familiar with comparing numbers in math using phrases such as "greater than," "less than," or "less than or equal to". Python also contains similar comparators that we can use to compare numbers or strings.

Comparison operators are used to compare two values. Python will return a Boolean value of either True or False after using a comparison operator. This output indicates whether a comparison is true or false.

Operator	Name	Example
==	Equal	3 == 3
!=	Not Equal	13 != 5
>	Greater than	120 > 23
<	Less than	20 < 50
>=	Greater than or equal to	60 >= 60
<=	Less than or equal to	75 <= 75

Conditional statements

Consider the following scenario:

"Rahul wants to play outside, but he cannot do so if it is raining. His decision to play outside depends on whether it is raining or not."

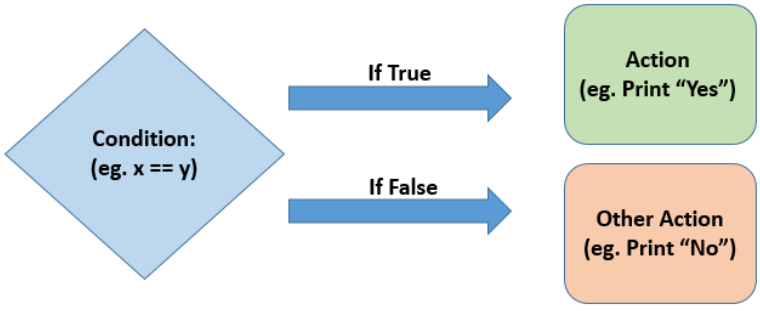
For a scenario like this, where our output depends on a condition, we use conditional statements.

Typically, in Python, a program is executed in a sequence. The first line is executed first, followed by the second line, and so on until it reaches the end of the program. Sometimes, we need to execute a specific part of the program only if a condition is true. Here, we use conditional statements.

Conditional statements in Python

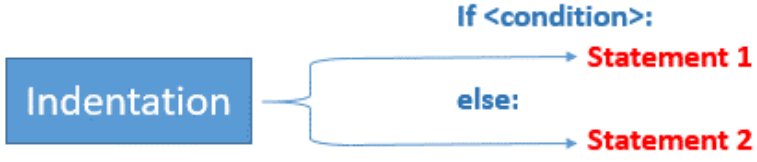
a. If-else block:

The if-else block is used to check for a condition which decides what happens next. If the condition is True, we will show Output 1 else we will show Output 2.



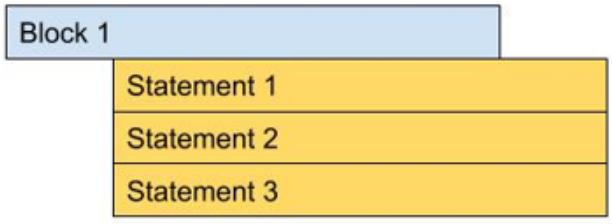
From the figure above, we know that there are 2 actions. Depending on whether the condition is true or false, the corresponding action will be performed.

Syntax for if-else block:

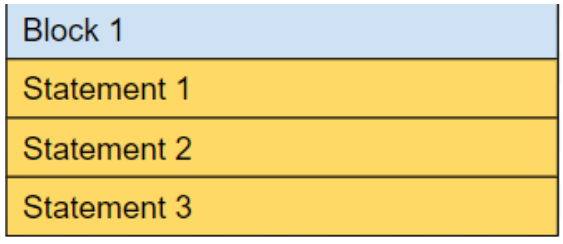


Indentation in Python

Indentation refers to the spaces used at the beginning of a statement to indicate which instructions / statements belong together. The general rule for indentation is to use 4 spaces (which is achieved by pressing the tab key once).



In the figure above, Statement 1, 2 and 3 belong to Block 1 since they all have the same indentation.



In the next figure, Statement 1, 2, 3 do not belong to Block 1 as there is no indentation.

b. If-elif-else block:

Conditional logic can be more complex if we add more conditional statements. The if-elif-else statement enables us to set multiple conditions for Python to analyse before taking an action. The elif stands for “else if”.

Syntax for if-elif-else block:

```
if <condition>:
        Statement 1

elif <condition>:
        Statement 2

else:
        Statement 3
```

Byte

Syntax is a structure of statements or elements in a computer language.

Activity

Write a program to understand the if-else block.

We will write a program to take age in the form of a number as the user input. It will display 2 different messages depending on whether the age entered is greater than 18 or not.

1. Create a variable 'age' to store the age of person

```
if-else.py > ...
1   age = int(input("Enter your Age - "))
2
```

2. Use the if statement to specify a condition and associate a statement with it

```
age = int(input("Enter your Age - "))

if age >= 18:
    print("You are an Adult")
```

The condition checks if the value of the age variable is greater than or equal to 18. If this condition evaluates True, then "You are an Adult" is displayed.

3. If the condition evaluates to False, this means the value of the age variable is less than 18. Then, the else statement executes and displays "You are Not an Adult"

```
if-else.py > ...
1   age = int(input("Enter your Age - "))
2
3   if age >= 18:
4       print("You are an Adult")
5   else:
6       print("You are Not an Adult")
```

Exercise

State whether True or False

1. Python will return a Boolean value of either True or False after using a comparison operator.

   _____

2. The general rule for indentation is to use 2 spaces.

   _____

3. The if-elif-else statement enables us to set multiple conditions for Python to analyse before taking an action.

   _____

Activity

Create a program to take a number from the user and check if it is an even number or odd number.

1. Create a new file in VS Code and save it as Odd_Even.py
2. Create a variable “num”, which will store the input provided by the user

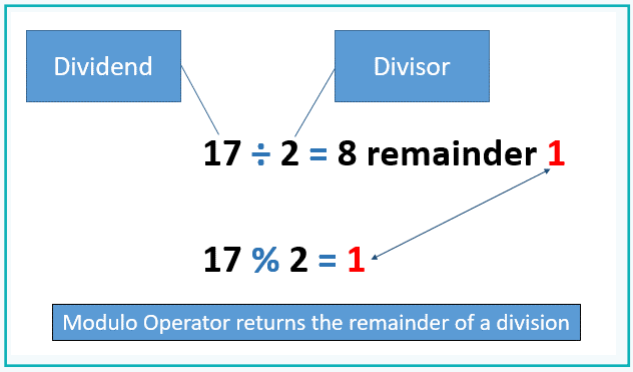
```
Odd_Even.py > ...
1    num = int(input("Please provide a number - "))
```

Activity

3. Create an if statement. This statement will contain a condition to check if the number provided by the user is Even

```
Odd_Even.py > ...
1   num = int(input("Please provide a number - "))
2
3   if num % 2 == 0:
4       print("The number is Even")
```

4. In the condition, we are using the % (modulus) operator which gives us a remainder



- num % 2 will give us a remainder after the value stored in the num variable is divided by 2.
- In num % 2 == 0, we compare if the remainder is equal to 0. This is done using == (equals to) operator which compares if two expressions/statements are equal or not.
- If the condition evaluates to 0, it means that the value in the num variable is divisible by 2, indicating that it is an even number.
- If the remainder is not equal to 0, the condition evaluates to False indicating that the number is not Even. In that case, our else block is executed.

```
Odd_Even.py > ...
1   num = int(input("Please provide a number - "))
2
3   if num % 2 == 0:
4       print("The number is Even")
5
6   else:
7       print("The number is Odd")
```

Activity

5 Execute the program using 'py Odd_Even.py'

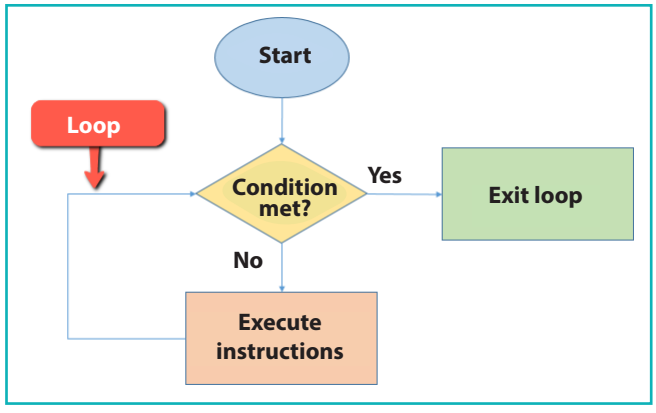
This is what the final output will be, if there are no errors:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
PS D:\Python\Programs> py Odd_Even.py
Please provide a number - 20
The number is Even
PS D:\Python\Programs> py Odd_Even.py
Please provide a number - 7
The number is Odd
PS D:\Python\Programs>
```

This program will run once and then stop. If we want to check if another number is odd or even, we will have to run the program again. We can use loops to repeatedly execute the instructions without having to manually run the program again.

Loops

A major reason why we use computers so much is because they are better at doing the same task multiple times, unlike humans who get tired or make mistakes. As we have learned in earlier lessons, a loop is used in programming to execute a set of instructions repeatedly until the specified condition is met. Each cycle where instructions are executed in a loop is called an iteration.



Types of Loops

There are 2 types of commonly used loops in programming:

a. For loop
b. While loop

For Loop

For loop repeats a block of code a fixed number of times. The “for” loop in Python is used to iterate over a series of iterable objects.

For example: string of values.

Syntax:

```
for item in object:
    Statements
```

a. **‘item’** is the value taken from the iterable one by one.
b. **‘object’** is a series of iterables.
c. **‘Statements’** are the instructions that execute once per iteration.

For example: If we want to print each letter of a word one by one, we can use ‘for’ loop:

```
1   for item in "Python":
2       print(item)
```

Output:

```
P
y
t
h
o
n
```

The code starts at the first item (the first letter) in the string “Python” and prints the item. The loop then repeats the same action for the next item. The loop continues to repeat until all the items in the string are printed.

While loop

A while loop repeats (or iterates) all instructions given under the while loop, as long as the defined condition holds true.

Syntax:

```
while expression:
        Statements
```

For example: If we want to print all numbers up to 10 one by one, we can use the while loop as follows:

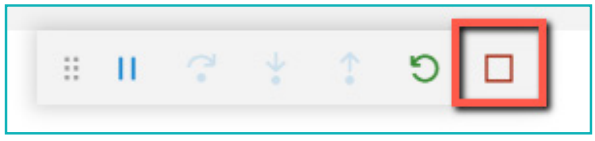
```
1    i = 1
2
3    while i<=10 :
4        print(i)
5        i = i + 1
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Remember to increment 'i' using (i = i + 1 or i += 1 statement), or else the loop will run forever i.e. in an infinite loop.

If the loop goes into an infinite loop in VS code, press the stop button as shown:



Exercise

Fill in the blanks

4. _____ operator compares two expressions/statements to see if they are equal or not.

5. A while loop repeats (or iterates) all instructions given under the while loop as long as the defined condition is _____.

6. Each cycle of execution of the instruction inside the loop is called an _____.

Activity

Create a program that takes numbers from the user and checks if it is an even number or odd number, repeatedly until the user provides "0" as an input.

1. Open the Odd_Even.py file in VS Code

2. Add the "while" statement at the start of the program with "True" as an expression

- The "True" expression will always make the while loop condition true, so the while loop will execute continuously.
- To keep the rest on the program inside while loop, add indentation to each line of code as shown:

```
1   while True:
2       num = int(input("Please provide a number - "))
3       if num % 2 == 0:
4           print("The number is Even.")
5       else:
6           print("The number is Odd.")
```

3. As the while loop is set to execute continuously forever, to stop the execution we need to add a "Break" statement when the user inputs '0'

- With the break statement we can stop the loop even if the while condition is true.
- The break statement can be used in both, while and for loops.

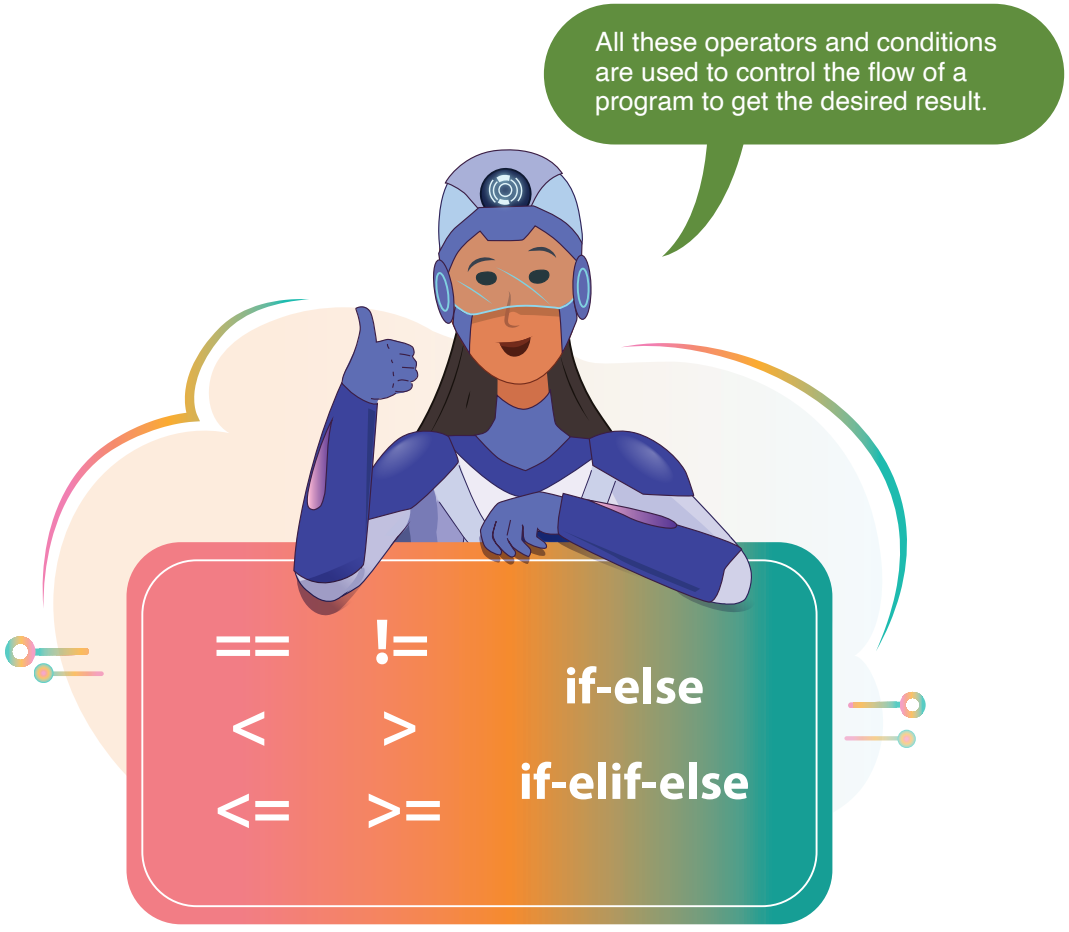
```
1   while True:
2       num = int(input("Please provide a number - "))
3       if num % 2 == 0:
4           print("The number is Even.")
5       else:
6           print("The number is Odd.")
7
8       if num == 0:
9           break
10
```

Output:

```
Please provide a number - 5
The number is Odd.
Please provide a number - 8
The number is Even.
Please provide a number - 0
The number is Even.
```

14. Control Flow

Computers perform data analysis based on inputs given by the users. Multiple programs can be used to perform this analysis. Different operators such as if-else, if-elif-else can be used to analyse data effectively.



Tech Flash

- **Comparison operators** : Comparison operators are used to compare two values.
- **Conditional statements** : When a specific part of program needs to be executed based on whether a specified condition is true or false, a conditional statement is used.
- **Loop** : A loop is a structure used to execute a set of instructions repeatedly until the specified condition is met.
- **Break statement** : "Break" statement is used to stop the execution of the loop even if the while condition is true.